

→ n elements in array A

q queries.

→ $[l, r]$

→ $\text{Sum}[l \dots r]$

$\Sigma_{x1} \text{ arr}[i] = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \begin{bmatrix} -3 & 6 & 2 & 4 & 5 & 2 & 8 & -9 & 3 & 1 \end{bmatrix} \end{matrix}$

⇒ Queries. = (9)

L	R	ans
4	8	9
3	7	10

Brute force : For each query q ,
loop from l to r .

Tc: $O(nq)$ Sc: $O(1)$

⇒

1	2	3	4	5	6	7	8	9	10
2	8	14	29	31	49	65	79	88	97
2	6	6	15	2	18	16	14	9	9

$$\begin{aligned}\text{Runs scored in 7}^{\text{th}} \text{ overs} &= \text{Score}[7] - \text{Score}[6] \\ &= 65 - 49 \\ &= \underline{\underline{16}}\end{aligned}$$

$$\begin{aligned}\text{Runs scored from 6}^{\text{th}} \text{ to 10}^{\text{th}} \text{ over} &= \text{Score}[10] - \text{Score}[5] \\ &= 97 - 31 \\ &= 6\end{aligned}$$

$$\begin{aligned}\text{Runs scored from 3}^{\text{th}} \text{ to 6}^{\text{th}} \text{ over} &= \text{Score}[6] - \text{Score}[2] \\ &= 49 - 8 = 41\end{aligned}$$

$$pf[i] = \text{Sum}[0 \dots i]$$

$$arr[] = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ -3 & 6 & 2 & 4 & 5 & 2 & 8 & -9 & 3 & 1 \end{matrix}$$

$$pf[] = \begin{matrix} -3 & 3 & 5 & 9 & 14 & 16 & 24 & 15 & 18 & 19 \end{matrix}$$

$$Q = [l, r] \Rightarrow pf[r] - pf[l-1]$$

$\Rightarrow$ Build the prefix sum array.

$Tc: O(n)$
 $Sc: O(n)$
 $\underline{\underline{\quad}}$

```

int pf[n]:
pf[0] = arr[0]
for (int i = 1; i < n; i++) {
    pf[i] = pf[i-1] + arr[i];
}
    
```

$$\begin{matrix}
 & 0 & 1 & 2 & 3 & 4^{(i)} \\
 arr[] = & 3 & 6 & 8 & 1 & 4 \\
 pf[] = & 3 & 9 & 17 & 18 & 22
 \end{matrix}$$

```
for (int i = 0 ; i < 9 ; i++) L.
```

```
int ans;
```

if ($l == 0$) $\mathcal{L}$
ans = pf [x].

3 else 2.

$$\text{ans} = \text{pf}[x] - \text{pf}[l-1];$$

3

$$T_C = O(n + q)$$

SL: $O(n)$

14. Problem 2 Sum of even indexed elements

Given an array of size N and Q queries with start (s) and end (e) index. For every query, return the sum of all **even indexed elements** from s to e.

Example

A[] = { 2, 3, 1, 6, 4, 5 }

Query :

1 3 = 1

2 5 = 5

0 4 = 7

3 3 = 0

A[] = [2, ~~3~~, 1, ~~6~~, 4, ~~5~~]

pf[] = [2, 2, 3, 3, 7, 7]

l r

$$1 \quad 3 = pf[3] - pf[0] \\ = 3 - 2 = 1$$

$$0 \quad 4 = pf[4] = 7$$

$$2 \quad 5 = pf[5] - pf[1] \\ = 7 - 2 = 5$$

```
int pf[n]:  
pf[0] = arr[0]
```

```
for (int i = 1; i < n; i++) {
```

```
    if (i % 2 != 0)
```

```
        pf[i] = pf[i-1];
```

```
    else  
        pf[i] = pf[i-1] + arr[i];
```

```
}
```

```
for (int i = 0; i < 9; i++) {
```

```
    int ans;
```

```
    if (l == 0) {  
        ans = pf[r];
```

```
    } else {
```

```
        ans = pf[r] - pf[l-1];
```

```
}
```

19. Problem 3 Count of Pairs ag

Given a string s of lowercase characters, return the **count of pairs** (i, j) such that $i < j$ and $s[i]$ is 'a' and $s[j]$ is 'g'.

Example 1

```
String s = "abegag"
Ans = 3
```

$i \quad j$
0 3 = ag
0 5 = ag
4 5 = ag

0 1 2 3 4 5 6
a c g d g a g
 $i \quad j$

0 2 = ag
0 4 = ag
0 6 = ag
5 6 = ag
} = 4

0 1 2 3 4 5 6 7
b c a g g a a g
 $i \quad j$

2 3
2 4
2 7
5 7
6 7
} ag $\Rightarrow$ 5

Pseudocode

```
function count_ag(str) {  
    result = 0;  
    for (i -> 0 to n-1) {  
        if(str[i] == 'a'){  
            for (j -> i+1 to N-1){  
                if(str[j] == 'g') {  
                    result++;  
                }  
            }  
        }  
    }  
    return result;  
}
```

$$Tc: O(n^2)$$

$$Sc: O(\underline{1})$$

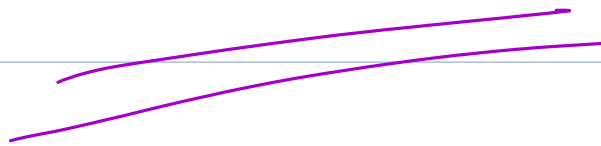


0 1 2 3 4 5 6

a c g d g a g

cg = 3 3 3 2 2 1 1

$$\text{ans} = \underline{1} + 3 = 4$$



0 1 2 3 4 5 6 7

b c a g g a a g

cg 3 3 3 3 2 1 1 1

$$\text{ans} = \underline{1} + \underline{1} + 3 = 5$$



	0	1	2	3	4	5	6	7
	b	c	a	g	g	a	a	g
ca	0	0	1	1	1	2	3	3

Result = 1 + 1 + 3

⇒ 5

Pseudocode

```
function count_ag(str) {
    result = 0;
    count_a = 0;
    for(i -> 0 to n-1) {
        if(str[i] == 'a') {
            count_a++;
        }
        else if(str[i] == 'g') {
            result += count_a;
        }
    }
    return result;
}
```

Tc: $O(n)$

Sc: $O(1)$

Definition

A subarray is a contiguous part of an array. It is formed by selecting a range of elements from the array. A subarray can have one or more elements and must be a contiguous part of the original array.

Example

Consider the following array of integers:

4 1 2 3 -1 6 9 8 12 4, 1, 2, 3

- 2, 3, -1, 6 is a subarray of length 4.
- 9 is a subarray of single element.
- 4, 1, 2, 3, -1, 6, 9, 8, 12 is a subarray consisting of all the array elements.
- 4, 12 is **not** a subarray because loop does not count as subarray.
- 1, 2, 6 is **not** a subarray because the array elements must be contiguous.
- 3, 2, 1, 4 is **not** a subarray because order of the elements in a subarray should be same as in the array.

The contiguous + same-order rule, made visual:

0 1 2 3 4 5 6
⇒ 4 2 10 3 12 -2 15

Start end
0 [0, 6]

$$\begin{aligned} &\hookrightarrow 6 - 0 + 1 \\ &= \underline{\underline{7}} \end{aligned}$$

0 1 2 3 4 5 6
⇒ 4 2 10 3 12 -2 15

Start end
1 [1, 6]

$$\begin{aligned} &= 6 - 1 + 1 \\ &= 6 \end{aligned}$$

Problem Statement:

Given an array of integers, we need to print all possible subarrays of the array.

Example

Input: [1, 2, 3]

Output:

[1]

[1, 2]

[1, 2, 3]

[2]

[2, 3]

[3]

⇒
1
1 2
1 2 3
2
2 3
3

void generateSubarrays (int arr[], int n) {

for (int i = 0; i < n; i++) { // Fixing start point.

for (int j = i; j < n; j++) { // Fixing end point.

for (int k = i; k <= j; k++)
 print(arr[k])

}

print("\n");

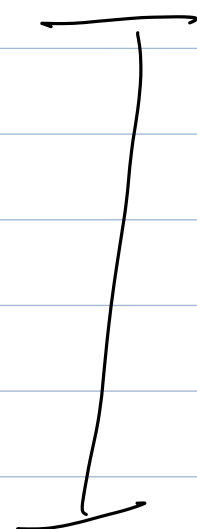
}

}

Tc: $O(n^3)$

$$arr [0 \quad 1 \quad \dots \quad n-1]$$

Start	end	total
0	$[0, n-1]$	n
1	$[1, n-1]$	$n-1$
2	$[2, n-1]$	$n-2$
$\vdots$		
$n-1$	$[n-1, n-1]$	1



$$= \frac{n(n+1)}{2}$$

—